Implementation of Continuous Integration and Continuous Delivery on AWS console using Managed AWS services

Architecture Overview:

GitHub → CodePipeline → CodeBuild → S3 Artifact → CodeDeploy → EC2

Here, Instead of Code commit we use GitHub as it is most widely used.

Step 1: Give below access to your IAM user and then login with IAM user account in to aws console.

AWSCodeBuildFullAccess

AWSCodePipelineFullAccess

AmazonSSMFullAccess

AWSCloudWatchLogsFullAccess

AWSCodeDeploy

Step 2: Create your GitHub repo with the application and buildspec.yml file for now, later we can modify them as per requirement.

AWS / DAY-08 / Sample-apache-app / **buildspec.yml**

sowmyasree33 Update buildspec.yml

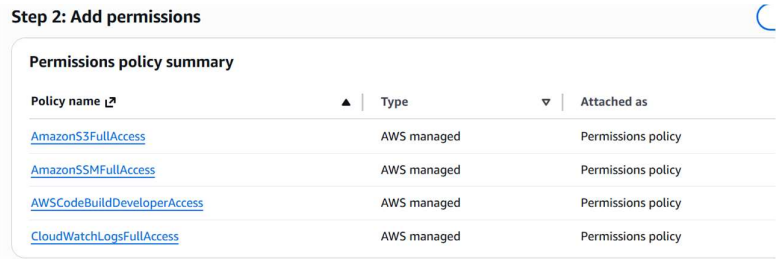
Code Blame

```
version: 0.2

phases:
  install:
    commands:
      - echo Installing Apache
      - sudo apt update
      - sudo apt install apache2 -y

  build:
    commands:
      - echo Moving to app directory
      - cd DAY-08/Sample-apache-app
      - ls

  post_build:
    commands:
      - echo Build completed
```

Step 3: Login with IAM user and search for code build and click on create project and mention all the required details as per images below.

Before that create service role required for code build to access aws services

Go to IAM->Roles->create role->attach following policies->create role



Step 2: Add permissions

Permissions policy summary

Policy name	Type	Attached as
AmazonS3FullAccess	AWS managed	Permissions policy
AmazonSSMFullAccess	AWS managed	Permissions policy
AWSCodeBuildDeveloperAccess	AWS managed	Permissions policy
CloudWatchLogsFullAccess	AWS managed	Permissions policy

Step 4: Now go for creating project in code build. Remember you have connect to your git hub repo. For that click on manage account credentials below source provider you will be navigated to your git hub repo. Give access and connect. After filling all the required details click create project.

Developer Tools > CodeBuild > Build projects > Create build project

Create build project

Project configuration

Project name

apache-app-sowmyab

A project name must be 2 to 255 characters. It can include the letters A-Z and a-z, the numbers 0-9, and the special characters - and _.

Project type

Select what type of project you would like to create. Info

- (●) **Default project** — Create a custom CodeBuild project.
- ( ) **Runner project** — Create a CodeBuild managed runner for workflows in GitHub Actions, GitHub Enterprise Actions, GitLab, or Buildkite.

▸ **Additional configuration**
Description, public build access, build badge, concurrent build limit, tags

Source

Add source

Source 1 - Primary

Source provider

GitHub

Credential

✓ Your account is successfully connected through OAuth using CodeBuild managed token. Manage account credentials.

☐ Use override credentials for this project only

Developer Tools > CodeBuild > Build projects > Create build project

Repository

- (●) Repository in my GitHub account
- ( ) Public repository
- ( ) GitHub scoped webhook

https://github.com/sowmyasree33/AWS.git

Source version - *optional* Info

Enter a pull request, branch, commit ID, tag, or reference and a commit ID.

▸ **Additional configuration**
Git clone depth, Git submodules, Build status config

Primary source webhook events Info

Webhook - *optional* Info

☑ Rebuild every time a code change is pushed to this repository

Build type

- (●) **Single build** — Triggers single build
- ( ) **Batch build** — Triggers multiple builds as single execution

Comment approval

ALL_PULL_REQUESTS

Approver roles

Developer Tools > CodeBuild > Build projects > Create build project

Environment

Provisioning model Info

- (●) **On-demand** — Automatically provision build infrastructure in response to new builds.
- ( ) **Reserved capacity** — Use a dedicated fleet of instances for builds. A fleet's compute and environment type will be used for the project.

Environment image

- (●) **Managed image** — Use an image managed by AWS CodeBuild
- ( ) **Custom image** — Specify a Docker image

Compute

- (●) **EC2** — Optimized for flexibility during action runs
- ( ) **Lambda** — Optimized for speed and minimizes the start up time of workflow actions

Running mode

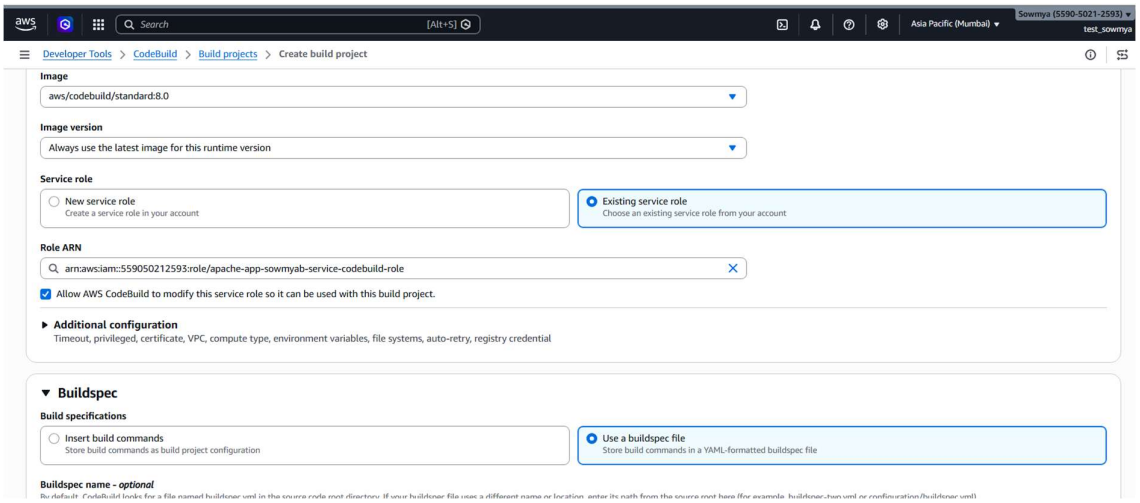
- (●) **Container** — Running on Docker container
- ( ) **Instance** — Running on EC2 instance directly

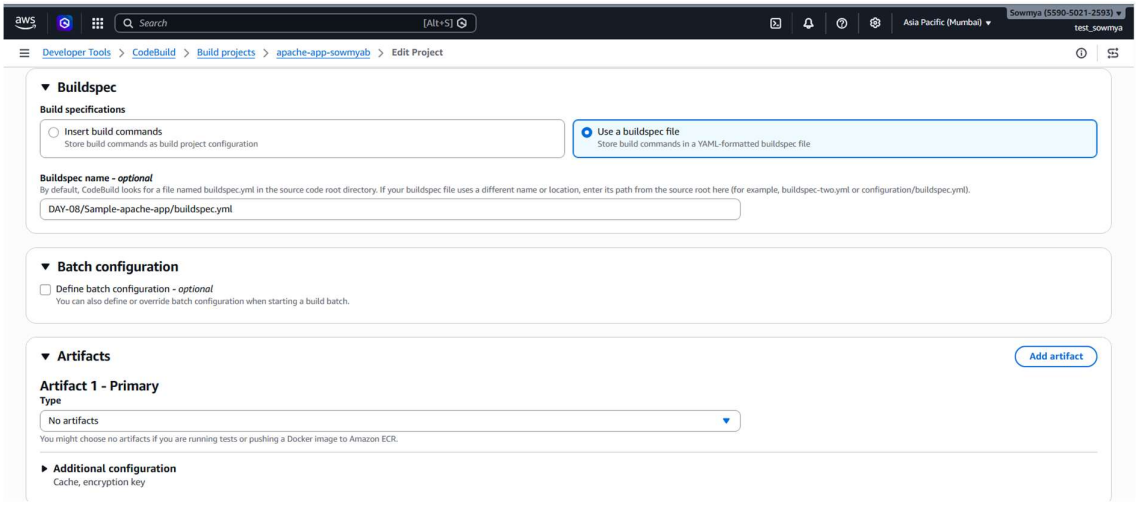
Operating system

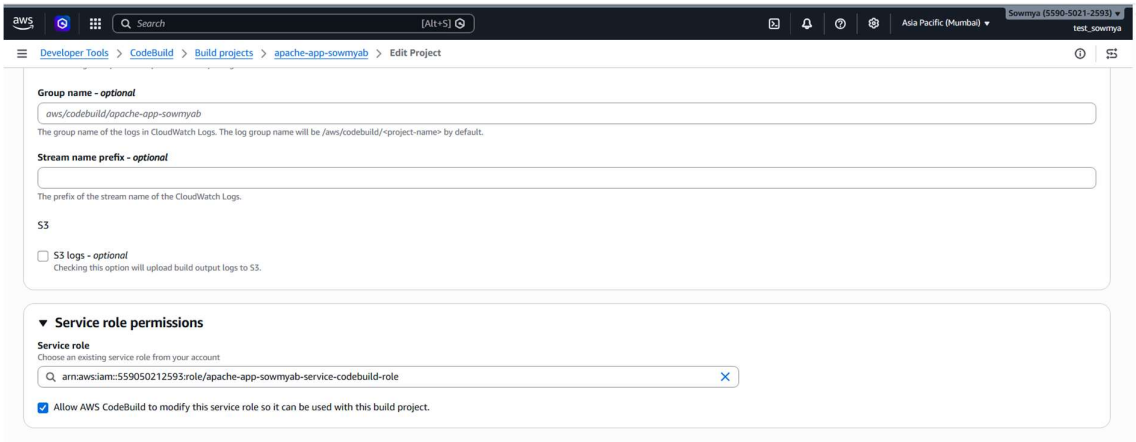
Ubuntu

Runtime(s)

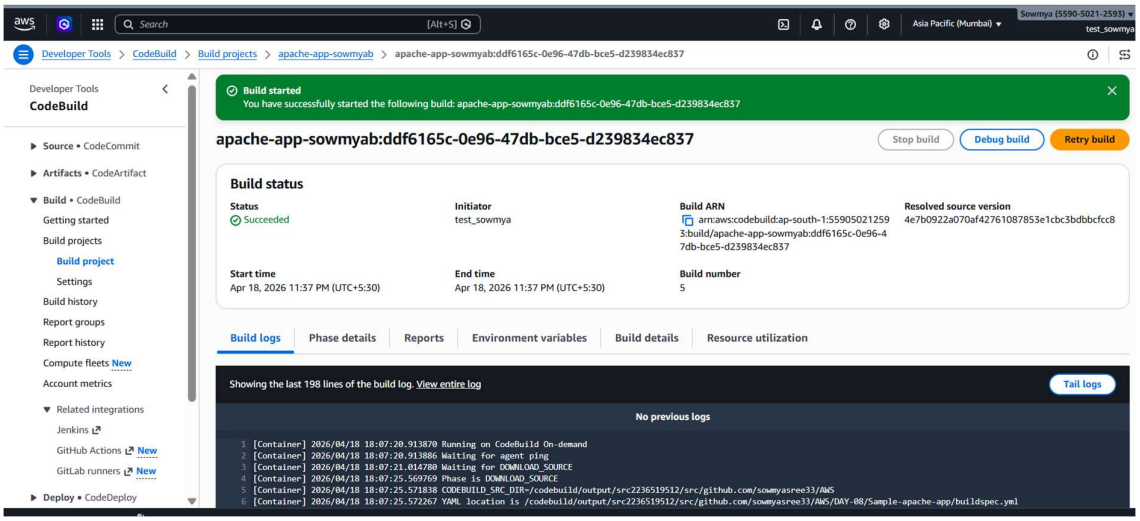
Standard







Step 5: Click on start build. You will see build success.



Step 6: Let's install docker on ubuntu using following commands

sudo apt update

sudo apt install docker.io -y

sudo systemctl start docker

sudo systemctl enable docker

sudo usermod -aG docker $USER (It adds your user to the docker group so that this user can directly use cmds like docker run , docker build without sudo)

newgrp docker

docker --version

docker run hello-world

Step 7: Create account in docker hub https://hub.docker.com

Step 8: Create docker file in your repo

Code Blame

```
FROM ubuntu:latest

RUN apt update && apt install apache2 -y

COPY index.html /var/www/html/

CMD ["apachectl", "-D", "FOREGROUND"]
```

Here you are installing Apache and copying file inside your docker container and last command to start Apache server when container runs.

Step 9: Now check try to access application by running docker container in ubuntu terminal with following commands

Prerequisite: clone your git hub repo from terminal (if already done ignore this)

```
cd ~/AWS/DAY-08/Sample-apache-app
docker build -t yourusername/apache-app .
docker run -p 8080:80 yourusername/apache-app
```

Also add below rule in your settings-> network->path forwarding

Name	Protocol	Host Port	Guest Port
web	TCP	8080	8080

Now access http://localhost:8080

Welcome to my app

You have successfully accessed to the application using docker container.

Step 10: let push image from terminal with below commands

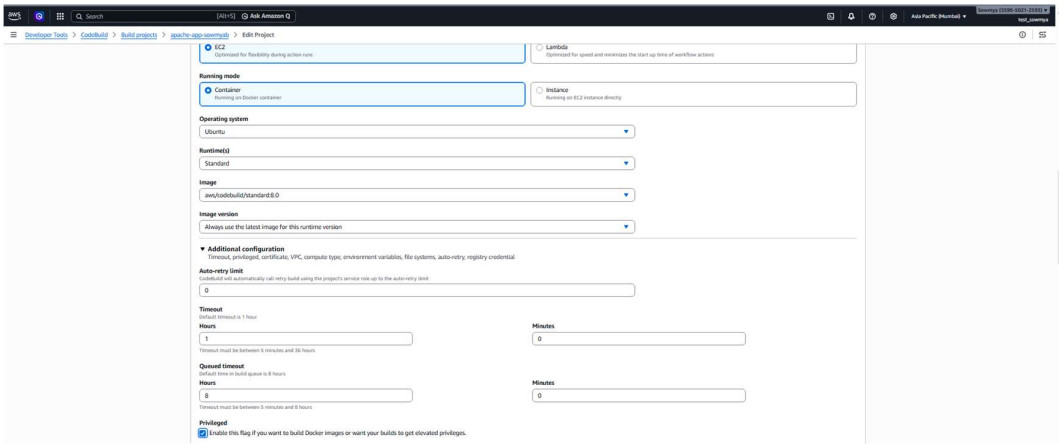
```
docker tag yourusername/apache-app yourusername/apache-app:latest
docker push yourusername/apache-app:latest
```

you will see image in your docker hub repository.

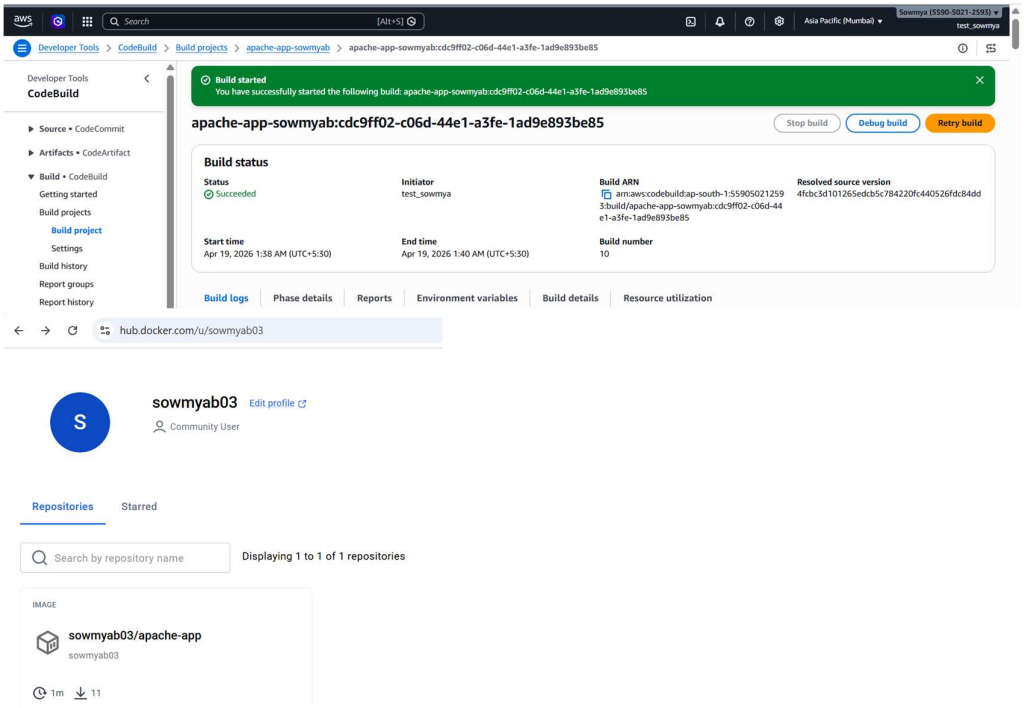
Step 11: Search for AWS System manager and store the docker login credentials.

Go to Application management->parameter store->create parameter

After that search code build->select project->edit->environment->additional config->enable privileged mode



Click on start build and you will be able to see the image in docker hub.



Step 12: Lets create code pipeline and integrate it with code build.

Search code pipeline->create pipeline->choose built custom pipeline

Skip test and deploy stage. Review everything then click on create pipeline.

aws | Search [Alt+S]

Developer Tools > CodePipeline > Pipelines > Create new pipeline

Step 2
Choose pipeline settings

Step 3
Add source stage

Step 4 - *optional*
Add build stage

Step 5 - *optional*
Add test stage

Step 6
Add deploy stage

Step 7
Review

Pipeline settings

Pipeline name
Enter the pipeline name. You cannot edit the pipeline name after it is created.

apache-app-sowmyab-codepipeline

No more than 100 characters

Execution mode Info
Choose the execution mode for your pipeline. This determines how the pipeline is run.

- ○ Superseded
- ◉ Queued
- ○ Parallel

Service role

- ◉ New service role — Create a service role in your account
- ○ Existing service role — Choose an existing service role from your account

Role name

AWSCodePipelineServiceRole-ap-south-1-apache-app-sowmyab-codepi

Type your service role name

☑ Allow AWS CodePipeline to create a service role so it can be used with this new pipeline

Advanced settings
Configure artifact store location, encryption settings, and pipeline variables for your pipeline.

aws | Search [Alt+S] | Asia Pacific (Mumbai) | Sowmya (5590-5021-2593) test_sowmya

Developer Tools > CodePipeline > Pipelines > Create new pipeline

Step 3
Add source stage

Step 4 - *optional*
Add build stage

Step 5 - *optional*
Add test stage

Step 6
Add deploy stage

Step 7
Review

Source provider
This is where you stored your input artifacts for your pipeline. Choose the provider and then provide the connection details.

GitHub (via OAuth app)

Grant AWS CodePipeline access to your GitHub repository. This allows AWS CodePipeline to upload commits from GitHub to your pipeline.

Connected

✓ You have successfully authenticated your account.

The GitHub (via OAuth app) action is not recommended
The selected action uses OAuth apps to access your GitHub repository. This is no longer the recommended method. Instead, choose the GitHub (via GitHub App) action to access your repository by creating a connection. Connections use GitHub Apps to manage authentication and can be shared with other resources. Learn more

Repository

sowmyasree33/AWS

Branch

main

☑ Enable automatic retry on stage failure

aws | Search [Alt+S] | Ask Amazon Q | Asia Pacific (Mumbai) | Sowmya (5590-5021-2593) test_sowmya

Developer Tools > CodePipeline > Pipelines > Create new pipeline

Step 1
Choose creation option

Step 2
Choose pipeline settings

Step 3
Add source stage

Step 4 - optional
Add build stage

Step 5 - optional
Add test stage

Step 6
Add deploy stage

Step 7
Review

Add build stage - *optional* Info
Step 4 of 7

Build - *optional*

Build provider
Choose the tool you want to use to run build commands and specify artifacts for your build action.

- ○ Commands
- ◉ Other build providers

AWS CodeBuild

Project name
Choose a build project that you have already created in the AWS CodeBuild console. Or create a build project in the AWS CodeBuild console and then return to this task.

apache-app-sowmyab ✕ or Create project

☐ Define buildspec override - *optional*
Buildspec file or definition that overrides the latest one defined in the build project, for this build only.

Environment variables - optional
Choose the key, value, and type for your CodeBuild environment variables. In the value field, you can reference variables generated by CodePipeline. Learn more

Add environment variable

Build type

- ◉ Single build — Triggers a single build.
- ○ Batch build — Triggers multiple builds as a single execution.

Region

Asia Pacific (Mumbai)

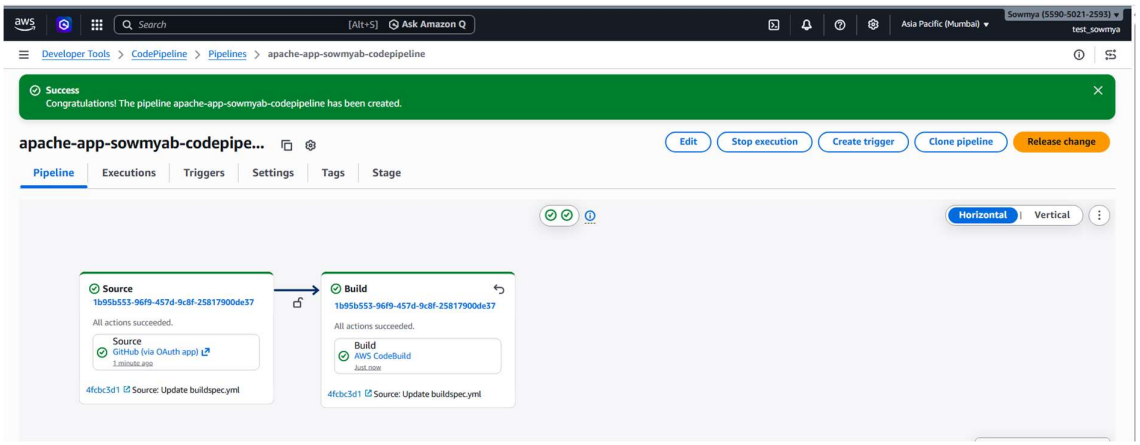
Input artifacts
Choose an input artifact for this action. Learn more

SourceArtifact ✕
Defined by: Source

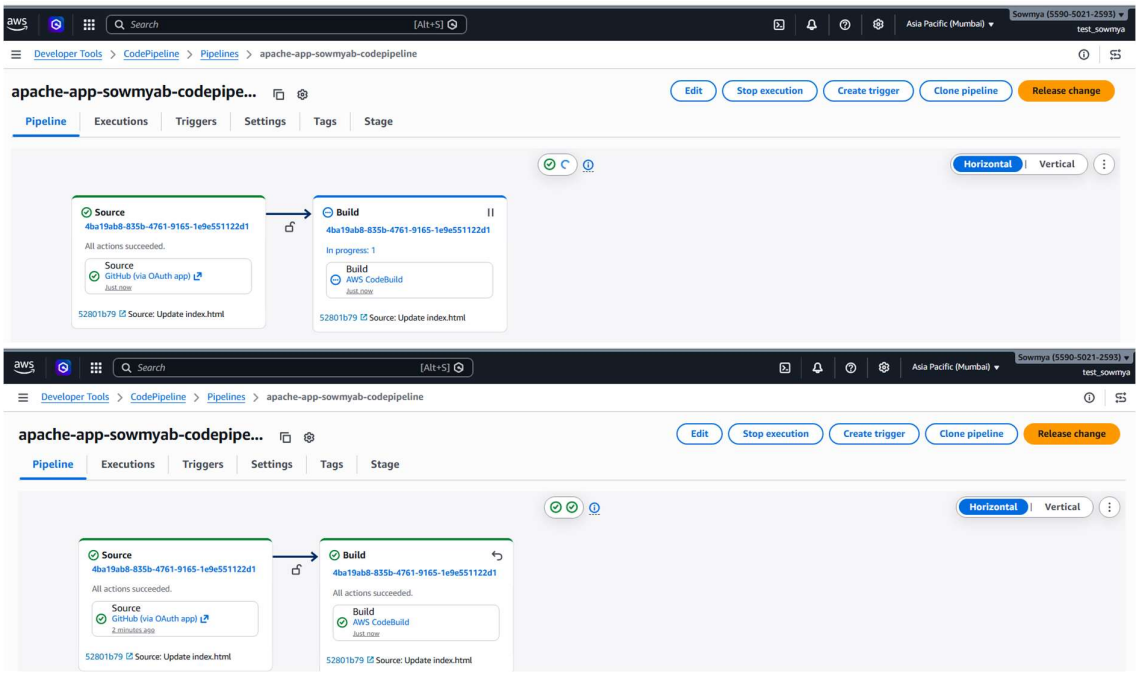
☑ Enable automatic retry on stage failure

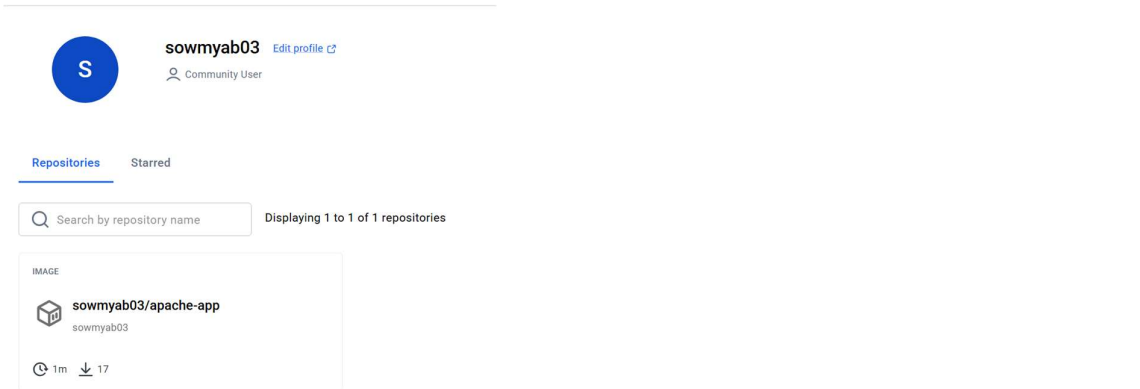
Cancel | Previous | Skip build stage | Next

Successfully integrated code pipeline with code build



Step 13: Let's verify this by modifying a file in GitHub repo. I have modified index.html file.



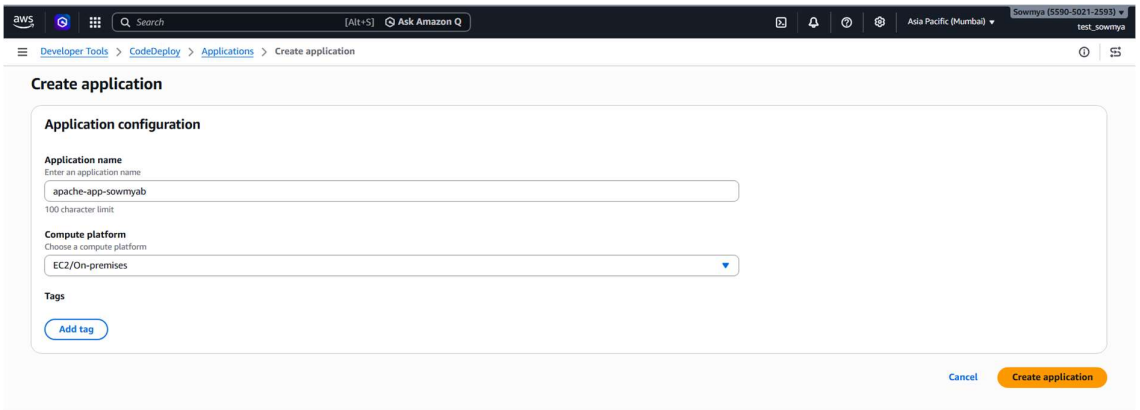


Note : Docker is used only for build demonstration, not for deployment in this pipeline.

Continuous Delivery

Step 1: Login to IAM user and search for AWS code deploy and create application.

I have given the same name which I used while creating project in code build.

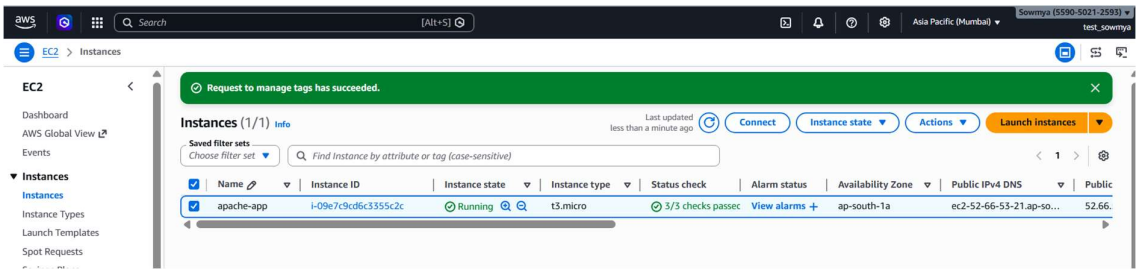


Step 2: Create EC2 application, while creating enable the auto-assign public ip and also add HTTP port 80(this is as per my application) so enable the network inbound rules as per your application.

Here I'm using default VPC.

You can also give a tag(this is useful when code deploy wants to connect to ec2)

Select the ec2 instance->actions->instance settings->manage tags



Step 3: Login to ec2 instance from the ubuntu terminal (to install code deploy agent in the ec2 instance)

AWS Code Deploy itself cannot directly log into or control your server. It needs a local execution component inside the instance to perform deployment tasks.

To login to your ec2 instance from ubuntu terminal(as we are using ubuntu from vm) follow below steps

1. Virtual box->settings->shared folders (inside this add the folder path)
2. Reboot vm and go to ls/media/sf_downloads. Here you will be able to see the .pem you have created.

3. Then connect to ec2, still if you are facing issue install the dependencies i.e on top of VM go to->device->insert guest additional cd image and ran following commands.
4. sudo apt update
5. sudo apt install build-essential dkms linux-headers-$(uname -r) -y
6. cd /media
7. ls

Now you will see the file and then connect to ec2 instance with below command

ssh -i name of pem file ubuntu@ public ip of instance

Step 4: Follow below document to install code deploy agent for ubuntu server

https://docs.aws.amazon.com/codedeploy/latest/userguide/codedeploy-agent-operations-install-ubuntu.html or use below commands

sudo apt update

sudo apt install ruby-full

sudo apt install wget

wget https://aws-codedeploy-ap-south-1.s3.ap-south-1.amazonaws.com/latest/install

chmod +x ./install

sudo ./install auto

sudo service codedeploy-agent status

Step 5: Create a role and attach it to ec2, as ec2 instance need to access the code deploy.

IAM->role->create role->aws service->select ec2->attach amazonec2forcodedeploy policy->click on create role

Search for EC2->select instance->actions->security->modify iam role.

Attach the role you have created and update the iam role

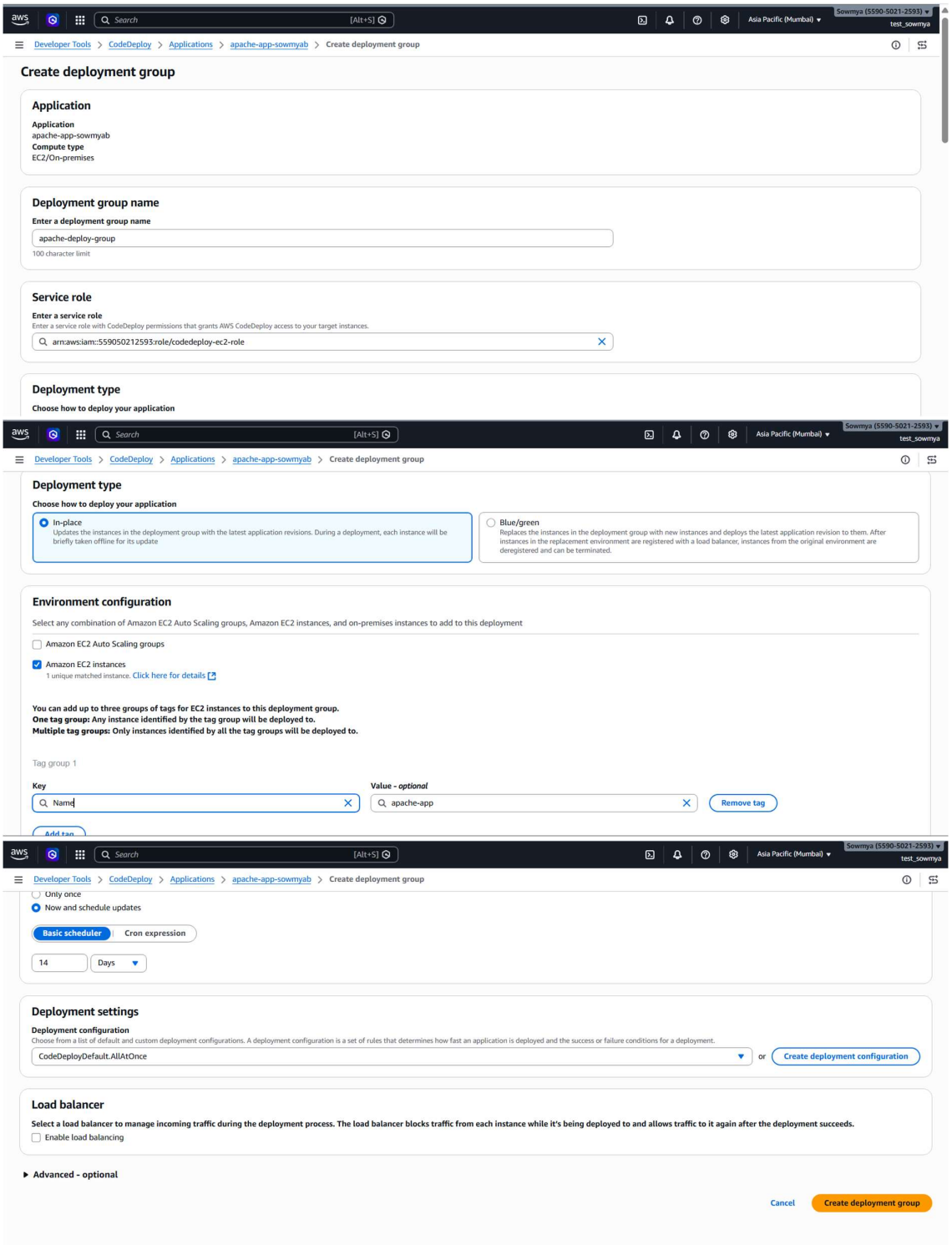
Whenever you make changes the ec2 instance(by attaching roles/modifying something) run below commands in terminal

Sudo service codedeploy-agent restart and to verify if agent is running or not also run the status command

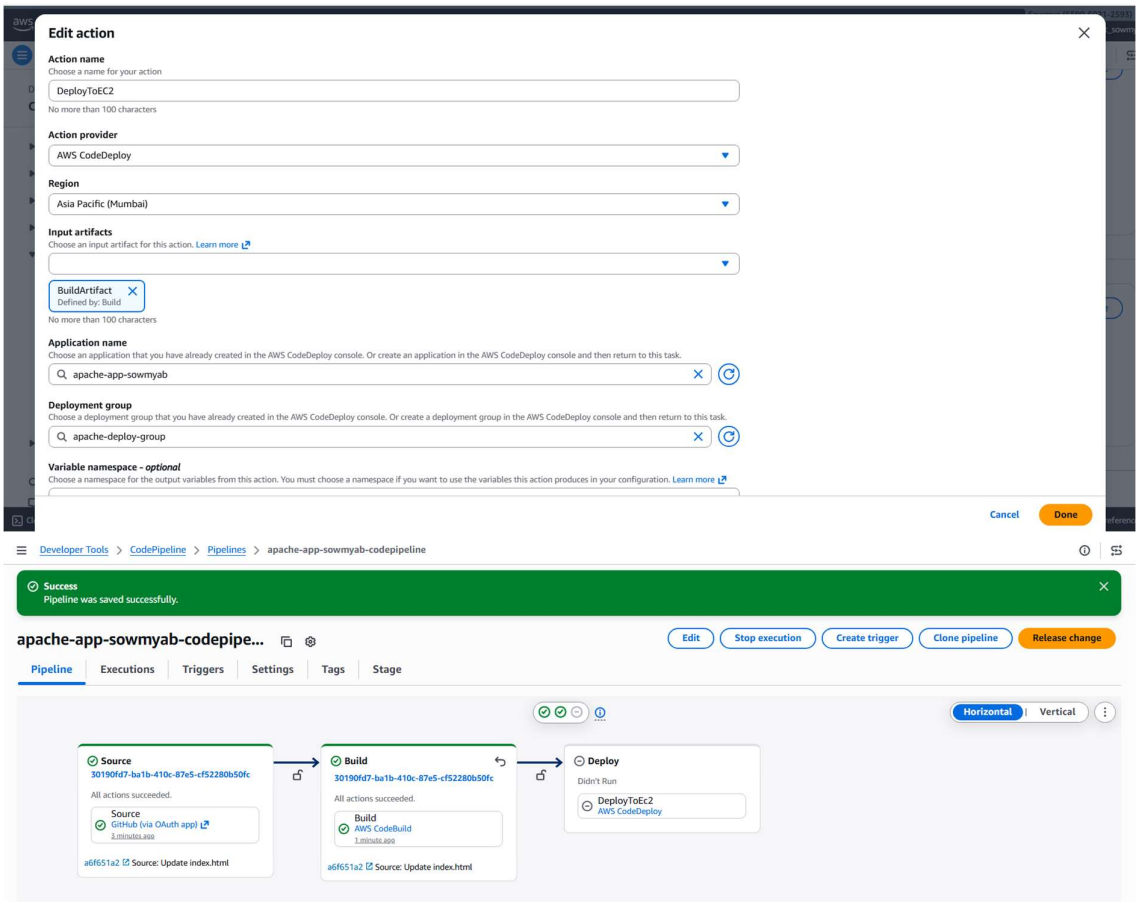
Step 6: Now create appsepc.yml(instruction file for code deploy) and install_dependencies.sh(to install required software on ec2) files in the git hub repo. you can refer to appsepc.yml and dependencies file present in this git hub repo.

Step 7: Search for Code deploy->select the application created->deployment groups->create deployment group->fill details->click on create deployment group

Before this create a service role by attaching awscodedeploy policy.



Step 8: Search for Code pipeline->edit->scroll down to bottom->click on add stage->give as deploy. Fill the details and click on done.Then after click on save on top right corner



Step 9:Go to pipeline and run. If u fail at deploy stage. Debug the error and fix it. Once error is fixed follow below steps to re-run .

Go to terminal->run the below commands

Sudo service codedeploy-agent restart

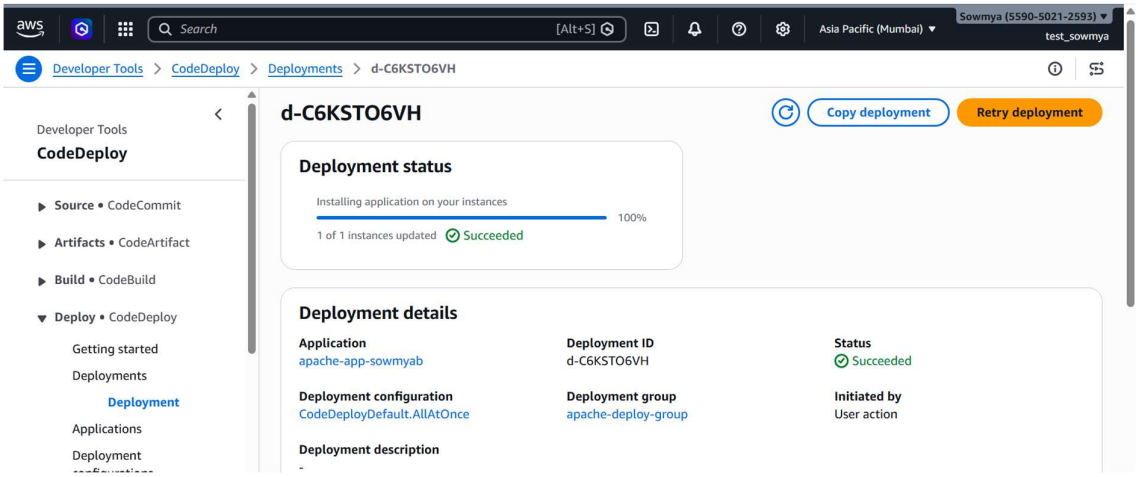
Sudo service codedeploy-agent status

Then go to console->click on release change on top right corner->release

The pipeline will rerun(takes some time to reflect)

Step 10: I failed at code deploy stage multiple times so I manually ran the code deploy, understood errors and debugged them and it got success now.

Here I used s3 instead of git hub i.e I have uploaded the appspec,indext.html and scripts in to them(as I was unable to connect to github)

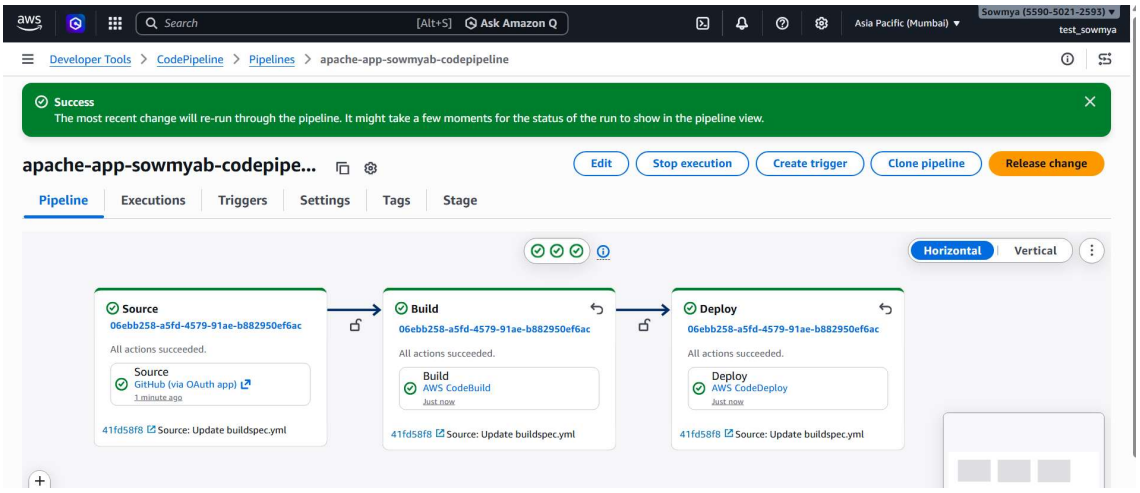


Step 11 : Check all the files in the gith hub repo, if everything is good and if not modify them, Search Code pipeline and check the status..

Even if it fails at deploy stage->go to iam->roles->for codeploy_ec2_role ->add code pipeline full access and also add inline policy for decryption(s3 defaults encrypts your file)

Check if you have added artifacts in buildspec.yml file.

If everything looks good, go to pipeline->click on release change->release



Hurray!! Successfully implemented CI/CD pipeline using AWS managed services.

Learned how to debug deployment failures and integrate build and deploy stages effectively.